

# **Optimization of a Classroom Schedule Display System in C++**

## **Context**

I developed a classroom schedule display system that shows the current subject, faculty member, and room number based on the time of day. The system retrieves schedule information and updates the display automatically for students and faculty.

## **Optimization Techniques Applied**

### **1. Replacing Sequential Search with Direct Lookup**

The initial design checked every schedule entry one by one to determine the current class. As the number of classes increased, the lookup time also increased.

To improve efficiency, schedule data was organized using structured records and indexed access techniques, allowing the program to locate the required schedule information more quickly.

### **2. Eliminating Redundant Display Updates**

The original implementation refreshed the display continuously, even when the subject and room information had not changed.

An optimization was introduced to update the display only when a schedule transition occurred. This reduced unnecessary processing and improved responsiveness.

### **3. Efficient Memory Usage**

Schedule information was stored using compact data structures instead of multiple independent variables. This reduced memory consumption and simplified data management.

## **Measurable Impact**

- Reduced the number of schedule comparisons required during lookup.
- Decreased processor workload by avoiding unnecessary display refreshes.
- Improved overall response time when switching between class periods.
- Enhanced scalability, allowing additional classrooms and schedule entries to be added with minimal performance impact.

## **Conclusion**

By optimizing schedule lookup, minimizing redundant display operations, and improving memory organization, the classroom scheduling system became more efficient, scalable, and responsive while maintaining accurate real-time schedule information.